

Assignment 6: Practical Application of A* Algorithm.

Instructions: This assignment engages you in the practical application of A* Algorithm. Follow the steps carefully, and submit your work as a single PDF file that includes all required code and graphs. **Attach** supplementary code files (.py and .ipynb) as used in this assignment.

Objective: Learn A* Algorithm and understand the effects of A* Algorithm in any problem.

Step-by-Step Instructions

Q1. A* algorithm

Task: Implement the A Algorithm for the 8-Puzzle Problem*

STEP 1:

Task Setup:

- Define a Node class to encapsulate the **state of the puzzle**, its **parent state**, and the cost: g, h, and f.
 - **g (cost):** Represents the cost from the start node to the current node, where each move incurs a cost of one.
 - **h (heuristic):** The estimated cost from the current node to the goal, calculated using the Manhattan Distance between the current state and the goal state.
 - **f (total cost):** The sum of g and h, determining the path priority in the A* algorithm.

Implementation Details:

- Start with the initial puzzle state: ((1, 2, 3), (4, 0, 6), (7, 5, 8)).
- Set the goal state as: ((1, 2, 3), (4, 5, 6), (7, 8, 0)).
- Implement the A* search to explore paths, expand nodes based on the lowest f value, and efficiently find the shortest path to the goal state.
- Attach your python code and your result that your get.

Main Function for Running:

```
def main():
    """Main function to run A* algorithm on a given 8-puzzle problem."""
    start = (
        (1, 2, 3),
        (4, 0, 6),
        (7, 5, 8)
    )

    end = (
        (1, 2, 3),
        (4, 5, 6),
        (7, 8, 0)
    )

    path, total_cost = astar(start, end)

    if path:
        print(f"Solution found in {len(path) - 1} moves with total cost {total_cost}:\n")
        for i, board in enumerate(path):
            print(f"Move {i}:")
            print_board(board)
    else:
        print("No solution found.")

if __name__ == '__main__':
    main()
```

Example code for Manhattan distance:

```
def manhattan_distance(start_state, goal_state):
    """Calculate the Manhattan Distance heuristic for the 8-puzzle."""
    distance = 0
    for num in range(1, 9): # Loop through numbers 1 to 8 (tiles of the 8-puzzle)
        # Find the position of the number in the start state
        x1, y1 = -1, -1
        for ix, row in enumerate(start_state):
            for iy, value in enumerate(row):
                if value == num:
                    x1, y1 = ix, iy
                    break
        if x1 != -1:
```

```
        break

    # Find the position of the number in the goal state
    x2, y2 = -1, -1
    for ix, row in enumerate(goal_state):
        for iy, value in enumerate(row):
            if value == num:
                x2, y2 = ix, iy
                break
        if x2 != -1:
            break

    # Calculate the Manhattan distance for this number
    distance += abs(x1 - x2) + abs(y1 - y2)
return distance
```

Result Expectation:

Move 0:

1	2	3
4		6
7	5	8

Move 1:

1	2	3
4	5	6
7		8

Move 2:

1	2	3
4	5	6
7	8	

STEP 2:

Testing:

Test the A* algorithm with alternative start points, such as: ((2, 5, 4), (0, 8, 3), (1, 7, 6)).

```
def main():
    """Main function to run A* algorithm on a given 8-puzzle problem."""
    start = (
        (2, 5, 4),
        (0, 8, 3),
        (1, 7, 6)
    )

    end = (
        (1, 2, 3),
        (4, 5, 6),
        (7, 8, 0)
    )

    path, total_cost = astar(start, end)

    if path:
        print(f"Solution found in {len(path) - 1} moves with total cost {total_cost}:\n")
        for i, board in enumerate(path):
            print(f"Move {i}:")
            print_board(board)
    else:
        print("No solution found.")

if __name__ == '__main__':
    main()
```

Result Expectation:

- Document how many moves were needed to reach the goal state for each test case.
- Include all code snippets, testing scenarios, and results in your report.

Submission Requirements:

- Submit your completed assignment as a PDF containing all necessary code snippets and graphs.
- Extensive explanations or theoretical discussions should be included in the main text of your PDF, not in code comments.
- Attach corresponding .py and .ipynb files containing executable code.
- Ensure all files are clearly labeled and organized.
- Late submissions will be subject to deductions according to the policy.

Evaluation Criteria: Your assignment will be graded based on the accuracy of your implementation, the clarity of your presentation, and the completeness of your submission. Specific attention will be given to how well you follow these guidelines:

- **Step-by-Step Explanation:** This assignment requires clear, step-by-step explanations. You must explicitly label and explain each step (e.g., Step 1, Step 2, Step 3, Step 4). Failure to provide detailed explanations for each required step will result in a proportional deduction in your score. For example, if the assignment requires four steps and you complete only three, this will result in a score of 75 out of 100.